



Spring Boot – JPA Example using JPA Query Languages

demo4-boot-jpa: Files required

1. MyBootApplication.java	Updated in Demo4
2. Customer.java	Updated in Demo4
3. CustomerDAO.java	Updated in Demo4
4. CustomerDAOImpl.java	Updated in Demo4
5. application.properties	Updated in Demo4
6. pom.xml	Same as Demo3

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.*;
import org.slf4j.*;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    static Logger logger = LoggerFactory.getLogger("MyBootApplication");

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        logger.debug("Demo4- run() Starts");
        logger.info("Demo4- run() Starts");
        logger.warn("Demo4- run() Starts");

        // 1. Customers By City1 - JPA-QL
        List<Customer> mylist1 = custDAO.getCustomersByCity1("Blore");
        for (Customer mycust : mylist1) {
            System.out.println(mycust);
        }
    }
}
```



```
logger.info("-----");
// 2. Customers By City2 - Native-QL
List<Customer> mylist2 = custDAO.getCustomersByCity2("Blore");
for (Customer mycust : mylist2) {
    System.out.println(mycust);
}

logger.info("-----");
// 3. Customers By City3 - Named - JPA-QL
List<Customer> mylist3 = custDAO.getCustomersByCity3("Blore");
for (Customer mycust : mylist3) {
    System.out.println(mycust);
}

logger.info("-----");
// 4. Customers By City4 - Named - Native-QL
List<Customer> mylist4 = custDAO.getCustomersByCity4("Blore");
for (Customer mycust : mylist4) {
    System.out.println(mycust);
}

logger.info("-----");
// 5. Customers By City - Criteria Queries
List<Customer> mylist5 = custDAO.getCustomersByCity("Blore");
for (Customer mycust : mylist5) {
    System.out.println(mycust);
}

logger.info("-----");
// 6. Customers By Email - Criteria Queries
List<Customer> mylist6 = custDAO.getCustomersByEmail("sri@jlc");
for (Customer mycust : mylist6) {
    System.out.println(mycust);
}

logger.info("-----");
// 7. Customers By City and Email - Criteria Queries
List<Customer> mylist7 = custDAO.getCustomers("sri@jlc","Blore");
for (Customer mycust : mylist7) {
    System.out.println(mycust);
}
System.out.println("Done!!!");
logger.debug("Demo4- run() ends");
logger.info("Demo4- run() ends");
logger.warn("Demo4- run() ends");
}
```



```
public static void main(String[] args) {

logger.debug("Demo4- main() Starts");
logger.info("Demo4 - main() Starts");
logger.warn("Demo4- main() Starts");

SpringApplication.run(MyBootApplication.class, args);

logger.debug("Demo4- main() Ends");
logger.info("Demo4 - main() Ends");
logger.warn("Demo4- main() Ends");

}
}
```

2) Customer.java

```
package com.jlcindia.springboot;

import javax.persistence.*;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Entity
@Table(name="mycustomers")

@NamedQuery(name = "customerByCity-A",
            query = "from Customer cust where cust.city=:mycity")

@NamedQuery(name="customerByCity-B",
            query="select * from mycustomers cust where cust.city=?1",
            resultClass = Customer.class)
public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="cid")
    private int cid;

    @Column(name="cname")
    private String cname;

    @Column(name="email")
    private String email;

    @Column(name="phone")
    private int phone;
}
```



```
@Column(name="city")
private String city;

public Customer() {}
public Customer(String cname, String email, int phone, String city) {
    this.cname = cname;
    this.email = email;
    this.phone = phone;
    this.city = city;
}

public Customer(int cid, String cname, String email, int phone, String city) {
    super();
    this.cid = cid;
    this.cname = cname;
    this.email = email;
    this.phone = phone;
    this.city = city;
}

//Setters and Getters
//toString()
}
```

3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.util.List;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface CustomerDAO {

    public List<Customer> getCustomersByCity1(String city);
    public List<Customer> getCustomersByCity2(String city);
    public List<Customer> getCustomersByCity3(String city);
    public List<Customer> getCustomersByCity4(String city);

    public List<Customer> getCustomersByCity(String city);
    public List<Customer> getCustomersByEmail(String email);
    public List<Customer> getCustomers(String email,String city);
}
```



4) CustomerDAOImpl.java

```
package com.jlcindia.springboot;

import java.util.List;
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import javax.persistence.criteria.CriteriaBuilder;
import javax.persistence.criteria.CriteriaQuery;
import javax.persistence.criteria.Predicate;
import javax.persistence.criteria.Root;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
@Transactional
public class CustomerDAOImpl implements CustomerDAO {

    @PersistenceContext
    EntityManager entityManager;

    public List<Customer> getCustomersByCity1(String city){
        String JPAQL = "from Customer cust where cust.city=:mycity";

        List<Customer> mylist = entityManager.createQuery(JPAQL, Customer.class)
        .setParameter("mycity", city)
        .setFirstResult(0)
        .setMaxResults(3)
        .getResultList();

        return mylist;
    }

    public List<Customer> getCustomersByCity2(String city){
        String SQL = "select * from mycustomers cust where cust.city=:mycity";

        List<Customer> mylist = entityManager.createNativeQuery(SQL, Customer.class)
        .setParameter("mycity", city)
        .setFirstResult(0)
        .setMaxResults(3)
        .getResultList();

        return mylist;
    }
}
```



```
public List<Customer> getCustomersByCity3(String city){

return entityManager.createNamedQuery("customerByCity-A")
.setParameter("mycity", city)
.setFirstResult(0)
.setMaxResults(3)
.getResultList();

}

public List<Customer> getCustomersByCity4(String city){

return entityManager.createNamedQuery("customerByCity-B")
.setParameter(1, city)
.setFirstResult(0)
.setMaxResults(3)
.getResultList();
}

public List<Customer> getCustomersByCity(String mycity){
CriteriaBuilder cbuilder = entityManager.getCriteriaBuilder();
CriteriaQuery<Customer> mycriteria= cbuilder.createQuery(Customer.class);

Root<Customer> myroot = mycriteria.from(Customer.class);
Predicate cityPredicate = cbuilder.equal(myroot.get("city"), mycity);
mycriteria.where(cityPredicate);

return entityManager.createQuery(mycriteria)
.setFirstResult(0)
.setMaxResults(3)
.getResultList();

}

public List<Customer> getCustomersByEmail(String myemail){
CriteriaBuilder cbuilder = entityManager.getCriteriaBuilder();
CriteriaQuery<Customer> mycriteria= cbuilder.createQuery(Customer.class);

Root<Customer> myroot = mycriteria.from(Customer.class);
Predicate emailPredicate = cbuilder.equal(myroot.get("email"), myemail);

mycriteria.where(emailPredicate);

return entityManager.createQuery(mycriteria)
.setFirstResult(0)
.setMaxResults(3)
.getResultList();
}
```



```
public List<Customer> getCustomers(String email,String city){
CriteriaBuilder cbuilder = entityManager.getCriteriaBuilder();
CriteriaQuery<Customer> mycriteria= cbuilder.createQuery(Customer.class);

Root<Customer> myroot = mycriteria.from(Customer.class);
Predicate emailPredicate = cbuilder.equal(myroot.get("email"), email);
Predicate cityPredicate = cbuilder.equal(myroot.get("city"), city);
mycriteria.where(emailPredicate,cityPredicate);

return entityManager.createQuery(mycriteria)
.setFirstResult(0)
.setMaxResults(3)
.getResultList();
}
}
```

5)application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5
spring.datasource.username=root
spring.datasource.password=srinivas
```

#Log Properties

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n

logging.level.com.zaxxer=DEBUG
logging.level.org.hibernate=INFO
logging.level.org.springframework=INFO
logging.level.com.jlcindia=WARN
```



Spring Boot - JPA Example using Programatic Transactions

demo5-boot-jpa: Files required

1. MyBootApplication.java	2. Account.java
3. AccountDAO.java	4. AccountDAOImpl.java
5. InsufficientFundsException.java	6. InvalidAccountNumberException.java
7. application.yml	8. pom.xml

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    static Logger logger = LoggerFactory.getLogger("MyBootApplication");

    @Autowired
    AccountDAO accountDAO;

    @Override
    public void run(String... args) throws Exception {

        logger.debug("Demo5- run() Starts");
        logger.info("Demo5- run() Starts");
        logger.warn("Demo5- run() Starts");

        System.out.println(accountDAO.getBalance(101));
        System.out.println(accountDAO.getBalance(102));
        System.out.println(accountDAO.getBalance(103));

        System.out.println("-----");
        System.out.println(accountDAO.getBalance(101));
        accountDAO.deposit(101,5000);
        System.out.println(accountDAO.getBalance(101));
    }
}
```



```
accountDAO.withdraw(101,5000);
System.out.println(accountDAO.getBalance(101));

try {
    System.out.println(accountDAO.getBalance(103));
    System.out.println(accountDAO.getBalance(101));
    accountDAO.fundsTransfer(103,101,5500.0);
    System.out.println(accountDAO.getBalance(103));
    System.out.println(accountDAO.getBalance(101));
} catch (Exception ex) {
    System.out.println(ex);
}

System.out.println("Done!!!");
logger.debug("Demo5- run() ends");
logger.info("Demo5- run() ends");
logger.warn("Demo5- run() ends");

}

public static void main(String[] args) {
    logger.debug("Demo5- main() Starts");
    logger.info("Demo5 - main() Starts");
    logger.warn("Demo5- main() Starts");
    SpringApplication.run(MyBootApplication.class, args);
    logger.debug("Demo5- main() Ends");
    logger.info("Demo5 - main() Ends");
    logger.warn("Demo5- main() Ends");
}

}
```

2) Account.java

```
package com.jlcindia.springboot;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```



```
@Entity
@Table(name = "myaccounts")
public class Account {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "accno")
    private int accno;

    @Column(name = "atype")
    private String atype;

    @Column(name = "balance")
    private double balance;

    //Constrcutors
    //Setters and Getters
    //toString()
}
```

3) AccountDAO.java

```
package com.jlcindia.springboot;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public interface AccountDAO {
    public double getBalance(int accno);
    public void deposit(int accno,double amount);
    public void withdraw(int accno,double amount);
    public void fundsTransfer(int saccno,int daccno,double amount);
}
```

4) AccountDAOImpl.java

```
package com.jlcindia.springboot;

import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.TransactionDefinition;
import org.springframework.transaction.TransactionStatus;
import org.springframework.transaction.support.DefaultTransactionDefinition;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```



```
*/
@Repository
public class AccountDAOImpl implements AccountDAO {

    @PersistenceContext
    EntityManager entityManager;

    @Autowired
    PlatformTransactionManager txManager;

    @Override
    public double getBalance(int accno) {
        Account account = entityManager.find(Account.class, accno);
        double bal = 0.0;
        if(account!=null)
            bal = account.getBalance();
        else
            throw new InvalidAccountNumberException();

        return bal;
    }

    @Override
    public void deposit(int accno, double amount) {
        TransactionStatus ts=null;
        try {
            TransactionDefinition txDef =
            new DefaultTransactionDefinition(TransactionDefinition.PROPGATION_REQUIRED);
            ts=txManager.getTransaction(txDef); //Tx Begin

            //Ops Starts
            Account account = entityManager.find(Account.class, accno);
            if(account!=null) {
                account.setBalance(account.getBalance()+amount);
                entityManager.merge(account);
            }else {
                throw new InvalidAccountNumberException();
            }

            txManager.commit(ts); //Tx Ends

        }catch(Exception ex) {
            txManager.rollback(ts); //Tx Ends
        }
    }
}
```



```
@Override
public void withdraw(int accno, double amount) {
    TransactionStatus ts=null;
    try {
        TransactionDefinition txDef =
        new DefaultTransactionDefinition(TransactionDefinition.PROPGATION_REQUIRED);
ts =txManager.getTransaction(txDef); //Tx Begin

        //Ops Starts
Account account = entityManager.find(Account.class, accno);
        if(account!=null) {
            double currBal = account.getBalance();
            if(currBal>=amount+5000) {
                account.setBalance(currBal-amount);
entityManager.merge(account);
            }else {
                throw new InsufficientFundsException();
            }
        }else {
            throw new InvalidAccountNumberException();
        }
txManager.commit(ts); //Tx Ends

    }catch(Exception ex) {
txManager.rollback(ts); //Tx Ends
    }
}

@Override
public void fundsTransfer(int saccno, int daccno, double amount) {
    TransactionStatus ts=null;
    try {
        TransactionDefinition txDef =
        new DefaultTransactionDefinition(TransactionDefinition.PROPGATION_REQUIRED);
ts =txManager.getTransaction(txDef); //Tx Begin

        System.out.println(getBalance(daccno));
        deposit(daccno, amount);
        System.out.println(getBalance(daccno));
        withdraw(saccno, amount);

txManager.commit(ts); //Tx Ends
    }catch(Exception ex) {
txManager.rollback(ts); //Tx Ends
    }
}
}
```



5) InvalidAccountNumberException.java

```
package com.jlcindia.springboot;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class InvalidAccountNumberException extends RuntimeException {
}
```

6) InsufficientFundsException.java

```
package com.jlcindia.springboot;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class InsufficientFundsException extends RuntimeException {
}
```

7)application.yml

```
spring:
  datasource:
    driver-class-name: com.mysql.jdbc.Driver
    url: jdbc:mysql://localhost:3306/myjlcdb5
    username: root
    password: srinivas

logging:
  level:
    root: INFO
    com.zaxxer: DEBUG
    org.hibernate: INFO
    org.springframework: INFO
    com.jlcindia: WARN
```

8)pom.xml

```
<project .....>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo5-boot-jpa</artifactId>
```



```
<version>1.0</version>
<name>demo5-boot-jpa</name>
<description>Pragmatic TxS Example</description>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



Spring Boot - JPA Example using Declarative Transactions

demo6-boot-jpa: Files required

1. MyBootApplication.java	Same as Demo5
2. Account.java	Same as Demo5
3. AccountDAO.java	Same as Demo5
4. AccountDAOImpl.java	Updated in Demo6
5. InsufficientFundsException.java	Same as Demo5
6. InvalidAccountNumberException.java	Same as Demo5
7. application.yml	Same as Demo5
8. pom.xml	Same as Demo5

4) AccountDAOImpl.java

```
package com.jlcindia.springboot;

import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.Isolation;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
@Transactional
public class AccountDAOImpl implements AccountDAO {

    @PersistenceContext
    EntityManager entityManager;

    public double getBalance(int accno) {
        Account account = entityManager.find(Account.class, accno);
        double bal = 0.0;
        if(account!=null)
            bal = account.getBalance();
        else
            throw new InvalidAccountNumberException();
        return bal;
    }
}
```



```
@Transactional( isolation = Isolation.READ_COMMITTED,propagation = Propagation.REQUIRED)
public void deposit(int accno, double amount) {
```

```
//Ops Starts
Account account = entityManager.find(Account.class, accno);
if(account!=null) {
    account.setBalance(account.getBalance()+amount);
    entityManager.merge(account);
}else {
    throw new InvalidAccountNumberException();
}

}
```

```
@Transactional( isolation = Isolation.READ_COMMITTED,propagation = Propagation.REQUIRED)
public void withdraw(int accno, double amount) {
```

```
//Ops Starts
Account account = entityManager.find(Account.class, accno);
if(account!=null) {
    double currBal = account.getBalance();
    if(currBal>=amount+5000) {
        account.setBalance(currBal-amount);
        entityManager.merge(account);
    }else {
        throw new InsufficientFundsException();
    }
}else {
    throw new InvalidAccountNumberException();
}

}
```

```
@Transactional( isolation = Isolation.READ_COMMITTED,propagation = Propagation.REQUIRED)
public void fundsTransfer(int saccno, int daccno, double amount) {
```

```
System.out.println(getBalance(daccno));
deposit(daccno, amount);
System.out.println(getBalance(daccno));
withdraw(saccno, amount);

}
}
```



Q58) How many types of Query Languages supported by JPA?

Ans:

JPA supports 4 types of Query Languages like Hibernate

- 1) JPA QL:
- 2) Native Queries:
- 3) Named Queries:
- 4) Criteria Queries:

Q59) How to use JPA QL in JPA?

Ans:

```
String JPAQL=" from Customer cust where cust.email=?1";  
List<Customer> mylist= entityManager.createQuery(JPAQL)  
                                .setParameter(1, email)  
                                .getResultList();
```

Q60) How to use JPA QL in JPA?

Ans:

```
String SQL="select * from mycustomers where email=?";  
List<Customer> mylist= entityManager.createNativeQuery(SQL, Customer.class)  
                                .setParameter(1, email)  
                                .getResultList();
```

Q61) How to use NamedQueries in JPA?

Ans:

```
@NamedQuery(name = "customerByCity",  
            query = "from Customer cust where cust.city=:mycity")
```

```
List<Customer> mylist= entityManager.createNamedQuery("customerByCity")  
                                .setParameter("mycity", city)  
                                .getResultList();
```

Q62) How to use Criteria Queries in JPA?

Ans:

```
CriteriaBuilder cbuilder = entityManager.getCriteriaBuilder();  
CriteriaQuery<Customer> mycriteria= cbuilder.createQuery(Customer.class);
```

```
Root<Customer> myroot = mycriteria.from(Customer.class);  
Predicate emailPredicate = cbuilder.equal(myroot.get("email"), myemail);  
mycriteria.where(emailPredicate);
```

```
List<Customer> mylist=entityManager.createQuery(mycriteria).getResultList();
```



Q63) What are the Log Levels /Priorities Available?

Ans:

DEBUG, INFO, WARN, ERROR, FATAL

Q64) Is logger creates any performance issue in PROD Environment?

Ans:

Dev Mode : Use Debug

PROD Mode : INFO/WARN

Q65) What happens when I set INFO level for com?

Ans:

logging.level.com=INFO

Log Level for All the packages Starts with com will be set to INFO.

Q66) How to use YAML Files (Yet another Markup Language)?

Ans:

- Create the file called application.yml under src/main/resources
- Write the Required Properties in application.yml file

Q67) How Spring Boot scans for application.priperities or application.yml?

Ans:

Spring Boot scans for these files in the Classpath Resources.

If found in the classpath , You can see the following in Logs.

Origin: class path resource [application.yml]

Origin: class path resource [application.properties]

Q68) Can I use both application.priperities and application.yml?

Ans:

Yes, You can.

Q69) How can I use Declarative Transactions in Spring Boot?

Ans:

Nothing Special in Spring Boot. It is same as Spring Framework

@Transactional

Class AccountDAOImpl implements AccountDAO{

@Transactional(propagation = Propagation.REQUIRED)

public void fundsTransfer(int saccno, int daccno, double amount) {

//Core Logic Here

}

}



Q70) How can I use Pragmatic Transactions in Spring Boot?

Ans:

Nothing Special in Spring Boot. It is same as Spring Framework

```
TransactionStatus ts=null;
try {
    TransactionDefinition txDef = new DefaultTransactionDefinition(
        TransactionDefinition.PROPAGATION_REQUIRED);

    ts =txManager.getTransaction(txDef); //Tx Begin

    //Do your OPeratiosn Here

    txManager.commit(ts); //Tx Ends

} catch (Exception ex) {
    txManager.rollback(ts); //Tx Ends
}
```